

Digital Gram Panchayat

Project Report — What we built, why, and how

A bilingual e-governance portal for a Maharashtra Gram Panchayat

Prepared for project documentation. This report describes the complete Digital Gram Panchayat system as built to date: every functional module, the cross-cutting engineering (design system, accessibility, performance, security), and the reasoning and implementation approach behind each. It is written to be read by a reviewer or a future maintainer with no prior context.

Product	Digital Gram Panchayat (DGP) — citizen e-governance portal
Reference village	Sakharale, Taluka Walwa, District Sangli, Maharashtra
Stack	MERN — MongoDB/Mongoose, Express (ESM), React 18 + Vite 6 + Tailwind CSS
Structure	npm-workspaces monorepo: backend + citizen PWA + officer portal + shared
Languages	English and Marathi (full key parity)
Quality gate	247 automated tests, ESLint 0/0, backend coverage gate, both apps build

1. Executive summary

Digital Gram Panchayat is a production-oriented web platform that lets citizens of a village interact with their Gram Panchayat online — file complaints, apply for certificates (dakhala), read notices and government schemes, view and track property/water tax, receive notifications, and see who runs the village and how to reach them. Officers administer all of this from a separate portal: reviewing and approving applications, publishing notices and schemes, managing tax records, broadcasting alerts, editing the public village profile, and viewing audit and dashboard reports.

The system is delivered as two front-end applications — a mobile-first, installable Progressive Web App for citizens (offline-capable) and a desktop-first portal for officers — over a single Express/MongoDB API. Everything is bilingual (English + Marathi), token-themed for a professional government look, and hardened for scale and security.

Beyond features, substantial engineering went into the qualities a public portal needs: a unified design system, WCAG-oriented accessibility, database indexing and query safety for concurrency, authentication and upload hardening, an in-portal document viewer, and an automated test suite. Each of these is documented below with the reasoning behind it.

2. Architecture

The repository is an npm-workspaces monorepo so shared code lives in one place and both front-ends stay in step:

- **backend** — Express (ES modules) + Mongoose. Organised by feature folder (model / service / controller / routes / validation) so each domain is self-contained.
- **frontend/citizen-pwa** — React 18 + Vite, mobile-first PWA with offline support (Workbox service worker, IndexedDB offline queue), bottom-nav navigation.
- **frontend/admin-portal** — React 18 + Vite officer portal, sidebar + header layout, Recharts dashboards.
- **frontend/shared (@dgp/shared)** — the single source of truth for constants, validation rules, enums and small formatting helpers used by the API and both apps.

Cross-cutting engineering built into the backend: Zod-validated environment config that fails fast on bad settings; Helmet security headers with a strict Content-Security-Policy; strict CORS allow-list; request-correlation IDs; a provider-agnostic logger, error reporter and metrics layer; an append-only audit trail; idempotency keys for unsafe replays; liveness and readiness health checks; and graceful shutdown that drains in-flight requests.

Data lives in MongoDB, one collection per domain (users, complaints, notices, schemes, tax records, certificate applications, notifications, events, the village-profile singleton, audit logs, idempotency keys and counters). Uploaded media goes to Cloudinary in production, with a disk-backed local fallback for development.

3. Functional modules — what, why, how

Each module follows the same backend pattern and appears in both the citizen and officer experiences where relevant. The core modules below form the platform that later phases extended.

3.1 Authentication and roles

What	Citizen self-registration and login by mobile number; officer accounts are seeded (never self-registered). Two roles — citizen and officer — with role-gated routes.
-------------	--

Why	A government portal must separate the public from administrators, keep sessions safe, and let an account be disabled immediately.
How	Stateless JWTs (24h citizen / 8h officer) verified on every request, which re-loads the user and rejects disabled accounts. Passwords hashed with bcrypt. Per-IP and per-account login throttling blunts credential stuffing. (Later hardened — see §5.)

3.2 Complaints (grievances)

What	Citizens file categorised complaints (road, water, sanitation, electricity, other) with up to three photos and an optional GPS location; officers triage, change status and add notes; citizens track status on a timeline.
Why	Grievance redressal is a core Panchayat duty and needs an auditable, trackable trail.
How	Feature-folder API with a status history sub-document; photos validated and uploaded; a geospatial index on the location; offline submissions queue in the PWA and replay with an idempotency key so a reconnect never double-files.

3.3 Notices and government schemes

What	Officers publish notices and welfare schemes (with categories, attachments, publish/expiry control and auto-translation); citizens browse, search and filter them, online or offline.
Why	Public information — announcements and entitlements — must reach citizens in their language and remain readable without connectivity.
How	Published + active + date filters served from compound indexes; full-text search index on scheme title/summary/description; automatic English-to-Marathi translation fills the missing language; the PWA caches these for offline reading.

3.4 Tax records

What	Officers create property/water tax records for a citizen — assessed amount, financial year, due date — attach the scanned original tax document, and record payments; citizens see a demand-bill view with dues, a payment timeline, and the attached documents.
Why	Tax visibility and payment history reduce office visits and disputes; the original document gives the citizen the authoritative bill.
How	A tax record carries an embedded payments array and a bills array of uploaded scans; the balance and status are derived; documents open inside the app (see §4.5).

3.5 Certificates (Dakhala) and the generation system

What	Citizens apply for Residence, Birth, Death, 7/12 extract and other certificates, uploading the required documents; officers review, may correct fields, then approve — which generates an official PDF certificate with a QR code and a public verification page.
Why	Issuing certificates is a flagship Panchayat service; the output must look official and be verifiable by any third party without logging in.
How	Per-type required-field and required-document validation from a shared map; on approval the service mints a unique certificate number and an opaque verification id, renders a QR of the verification URL, and generates a branded A4 PDF (watermark, seal/signature slots, village branding). A public, rate-limited endpoint verifies a certificate by number or scanned QR and never leaks a 404.

3.6 Notifications

What	In-app notification centre with unread counts; per-channel delivery (in-app, and mock SMS/voice/email providers); officer broadcast to many recipients with delivery breakdown.
-------------	---

Why	Citizens need to know when their complaint or application status changes; officers need to reach the village at scale.
How	A notification per recipient with status tracking and retry; broadcasts fan out and are summarised by aggregation; delivery uses a provider-agnostic layer so real SMS/voice can be plugged in without code changes.

3.7 Officer dashboard, reports, users and audit

What	Officer dashboard with charts (complaints, certificates, tax) via Recharts; user management; and a read-only audit-log viewer.
Why	Administration needs a bird's-eye view and an accountability record of who changed what.
How	Aggregated counts feed lazy-loaded charts; every state-changing action across the system writes an append-only audit entry (actor, action, before/after), surfaced in the viewer.

3.8 Progressive Web App and offline

What	The citizen app installs to the home screen and works offline for reading reference data and queuing complaint submissions.
Why	Rural connectivity is intermittent; the portal must remain useful without a live network.
How	Workbox service worker with per-route caching strategies; an IndexedDB queue holds offline complaint submissions and replays them (idempotently) on reconnect via background sync.

4. Public village portal, design system and experience

A large phase turned the app from a login-first tool into a public-facing Gram Panchayat portal, then invested in the visual and interaction quality a government service should have.

4.1 Public village portal and dynamic Village Profile

What	Signed-out visitors land on a village home page (hero, statistics, services, latest notices/schemes, upcoming events, gallery, emergency contacts, map, footer) instead of a login form. Everything is driven by an officer-editable Village Profile .
Why	Any Gram Panchayat should be able to adopt the portal by editing data — not code — and citizens should find key information without an account.
How	A Village Profile singleton (identity, statistics, leadership, social, emergency contacts, gallery, and directory members) plus an Events module, all edited from the officer portal; the public page renders from the public read API, and empty sections simply do not appear.

4.2 Real initial content — Sakharale

What	The profile is seeded once with the real Sakharale village data (population 9,144, 1,949 families, area, literacy, SC/ST figures, PIN 415414, national emergency helplines).
Why	The demo must show a real, credible village rather than placeholder text — and the import must not invent facts.
How	An idempotent seed script imports only publicly available figures; unknown facts are left blank or marked “Not Available”, and after import the data is edited solely from the officer portal.

4.3 Gram Panchayat directory

What	A searchable directory of officials (Sarpanch, Deputy Sarpanch, Gram Sevak, Talathi, Village Development Officer, Police Patil, ward and committee members) with photo, designation, ward, phone, email, office hours, address, responsibilities, tenure and status; plus a compact “Officials” block on the citizen dashboard.
Why	Citizens frequently need to reach the right official; this replaces word-of-mouth with a single, always-current source.
How	An additive members array on the Village Profile, edited from the existing officer editor (no duplicate module); the citizen views offer search, category/designation/ward filters, sort, click-to-call/email, copy, and a maps link, with an older flat-leadership fallback.

4.4 Premium design system

What	A cohesive, government-grade visual language: royal-blue primary, soft #F5F7FA surfaces, white cards, an elegant navy dark theme, one type scale, layered elevation, consistent radii, subtle motion and shimmer loading states.
Why	The portal must read as an official, modern service — not a generic admin dashboard — and stay consistent across dozens of screens.
How	All colour, spacing, radius, shadow and type values are HSL/token variables in one place per app, surfaced through Tailwind; shared primitives (Card, Button, Input, Chip, Table, Dialog, EmptyState, Skeleton, PageHeader) consume the tokens, so restyling them lifts every page at once.

4.5 Responsive layout, accessibility and the in-portal document viewer

What	Adaptive layouts (single column on phones, multi-column grids on desktop), the citizen dashboard reorganised into an information hub with an events hero, a fixed page-shift bug, and a document viewer that opens images and PDFs inside the app.
Why	Every screen size must be usable with no overflow; the interface must meet accessibility expectations; and citizens should read any document without an external application.
How	A stable scrollbar gutter removes horizontal jump between pages; a first-visit language gate and switchers persist the choice and sync the HTML lang attribute for screen readers; an in-portal modal renders PDFs in an embedded frame (the public asset route relaxes frame headers to its own front-ends only) so nothing opens in a separate tab or app.

5. Production hardening — scale and security

Two focused phases addressed the qualities that matter once real citizens use the system concurrently. The report is deliberately honest that some scale properties are infrastructure, not code.

5.1 Scale readiness

- **Query safety (DoS guard):** every list endpoint clamps its page size through one shared helper, so a crafted `?limit=9999999` can no longer pull an entire collection into memory and stall the event loop.
- **Compound indexes:** the real filter-then-sort query shapes (a citizen's records newest-first, officer status filters, the published public feed, upcoming events) are indexed, so they use an index scan instead of a collection scan plus in-memory sort.
- **Connection pool and timeouts:** a bounded Mongoose pool with server-selection and socket timeouts, plus a deliberate index-build path (`npm run db:indexes`) for large production collections.
- **Documented honestly:** reaching ~5,000 concurrent users with headroom then depends on running multiple stateless instances behind a load balancer, a managed MongoDB replica set, and a shared (Redis) rate-limit store — captured in the scaling guide.

5.2 Security

- **JWT hardening:** tokens are signed and verified with a pinned algorithm (HS256) and a fixed issuer/audience, rejecting algorithm-confusion and cross-service token reuse.
- **Upload content validation:** beyond the (spoofable) client MIME type, the real leading bytes of each upload must match its declared type (JPEG/PNG/WEBP/PDF signatures).
- **Transport and headers:** Helmet with a strict Content-Security-Policy, HSTS in production, no-store on API responses, and a strict CORS allow-list of the two front-ends.
- **Injection and abuse:** Mongo-operator sanitisation, validation on every write, a UUID-guarded file store (no path traversal), per-feature rate limiting, and no stack-trace leakage in production. Passwords require at least 8 characters.
- **CSRF:** not applicable by design — authentication is a Bearer token in a header, not an ambient cookie.

Recommended next security work (documented, not yet built): refresh-token rotation with revocation, password-complexity/breach checks, officer 2FA, CAPTCHA, and virus-scanning of uploads.

6. Testing and quality

Quality is enforced automatically so regressions are caught before merge:

Layer	Tooling	Result
Backend (API, integration + unit)	Jest + Supertest + in-memory MongoDB	222 tests
Citizen PWA	Vitest + Testing Library	20 tests
Officer portal	Vitest + Testing Library	5 tests

Layer	Tooling	Result
End-to-end	Playwright (CI, with Mongo + web servers)	wired in CI
Lint	ESLint (0 errors / 0 warnings gate)	clean
Coverage	Jest threshold gate on the backend	above gate
Builds	Vite production build, both apps	green

Live end-to-end checks were also performed in a real browser for the highest-risk items — for example, confirming that an officer-uploaded PDF tax document opens inside the citizen app in an embedded frame with no new tab, and that the certificate verification page and language switching behave correctly.

7. Recent engineering log

The most recent phase of work (public portal, certificate generation, design system, directory, scale and security hardening, and the in-app viewer) spans 47 commits, grouped below by theme. Each was built additively — no existing API, authentication or schema was broken — committed with conventional-commit messages, and verified with lint, tests and builds.

Theme	Representative commits (newest first)
Public village portal	Village Profile + Events backend; public landing + guest routing; officer editors; About/Achievements/footer
Real content	Seed Sakharale profile from public census data
Certificate generation	PDF template overhaul; certificate number + QR + public verification API; officer edit-before-issue; verification page + tests
UI/UX and a11y	Scrollbar-gutter page-shift fix; language gate + switchers; sync html lang; adaptive widths across list pages
Premium design system	Layered elevation/surface/radii tokens; elevated Card + shimmer Skeleton; polished inputs and empty states
Citizen directory	Members schema; officials on dashboard + searchable directory; officer members editor
Scale hardening	Pagination clamp (DoS guard); compound indexes; pool/timeout config + deliberate index builds; scaling guide
Security hardening	JWT pinning; magic-byte upload validation; min password 8; security posture doc + tests
In-app document viewer	DocModal + DocumentViewer; tax bills, certificate documents and generated PDFs open in-portal

Delivery note: this recent work currently lives on a stack of feature branches that have been pushed but not yet merged to the main branch, each additive on the previous. Merging them (in order, or as one consolidation) is the outstanding release step. The earlier core modules (authentication, complaints, notices, schemes, tax, certificates, notifications, dashboard, PWA) are already on main.

8. Status and remaining work

Delivered and verified:

- All core citizen and officer modules, bilingual, with 247 passing automated tests.
- A public, data-driven village portal seeded with real Sakharale content.
- Official certificate generation with QR-based public verification.
- A cohesive premium design system, responsive layouts and an in-portal document viewer.
- Scale and security hardening of the API, with an honest scaling guide.

Honest remaining / recommended work:

- **Release:** merge the stacked feature branches into main.
- **Infrastructure for scale:** multi-instance deployment behind a load balancer, managed MongoDB replica set, and a Redis-backed shared rate-limit store.
- **Security follow-ups:** refresh-token rotation + revocation, officer 2FA, password-complexity/breach checks, CAPTCHA, and upload virus scanning.
- **Feature polish:** per-directory-member photo upload (currently a URL field), an events calendar/registration view, and continued per-page layout refinement.

This document reflects the system as implemented to date. It favours accuracy over optimism: where something is deployment-owned or not yet built, it is stated as such.